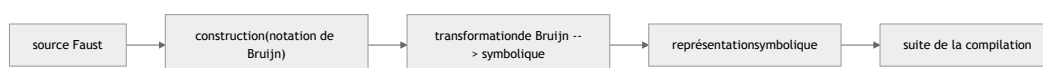


De Bruijn vers symbolique

Yann Orlarey
EMERAUDE (GRAMME/INRIA/INSA)
2026-06-12

Du de Bruijn au symbolique

Le compilateur Faust manipule des expressions récursives — boîtes et signaux bouclés sur eux-mêmes — qu'il doit **partager au maximum** : deux sous-expressions sémantiquement égales doivent être représentées par *le même* objet en mémoire. Ce partage repose sur le *hash-consing*[\[1\]](#), mais le hash-consing et la récursion ne cohabitent pas naïvement. Ce document explique pourquoi Faust utilise *deux* représentations des expressions récursives — la notation de de Bruijn[\[2\]](#) et la notation symbolique — et décrit la passe de transformation qui fait passer de l'une à l'autre.



Le partage et ses deux ennemis

Hash-consing

Technique de construction des termes garantissant que deux termes structurellement égaux sont représentés par un unique nœud physique. Chaque constructeur consulte une table globale : si le nœud demandé existe déjà, il est réutilisé. L'égalité sémantique se teste alors en temps constant (comparaison de pointeurs) et l'arbre syntaxique devient un DAG.

α -équivalence

Égalité de deux termes à renommage près des variables liées. `letrec x = f(x) et letrec y = f(y)` définissent le même signal mais ne sont pas structurellement égaux si les noms `x` et `y` figurent dans la représentation.

Terme clos

Terme sans variable libre — dont la dénotation ne dépend d'aucun environnement.

Le hash-consing identifie l'égalité **structurelle**. Pour qu'il serve l'égalité **sémantique**, il faut que la représentation soit telle que *structurellement égal* $\iff$ *sémantiquement égal*, du moins dans le sens utile. Deux obstacles se présentent, et chacune des deux représentations en résout exactement un :

1. l' α -équivalence — résolue par la notation de de Bruijn, qui élimine les noms ;

2. la dépendance au contexte des termes ouverts — résolue par la notation symbolique, où tout sous-terme est clos.

Les deux représentations

Notation de de Bruijn

Le lieu de récursion μ ne nomme pas sa variable ; une occurrence liée est un indice $[n]$ qui compte le nombre de μ à traverser pour atteindre son lieu.

$\mathbf{t} ::= k$	<i>constante, primitive</i>
$\mathbf{Op}(o, \mathbf{t}, \dots, \mathbf{t})$	$o \in \Omega$, opération n -aire
$\mathbf{Mu}(\mathbf{t})$	$\mu.t$ — définition récursive
$\mathbf{Ref}(n)$	$n \in \mathbb{N}$ — indice de de Bruijn

C'est une **forme canonique modulo α -équivalence** : deux définitions récursives égales à renommage près ont exactement la même représentation. Le hash-consing appliqué à la *construction* des termes de Bruijn identifie donc correctement les définitions récursives entre elles — ce qu'une représentation nommée naïve ne garantit pas.

À titre d'exemple, l'accumulateur — le plus petit programme Faust récursif — se note $\mu.(1 + [0])$:

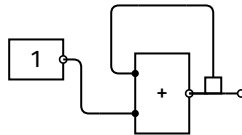


Figure 1: Accumulateur : $y(n) = 1 + y(n-1)$

Notation symbolique

Le lieu disparaît au profit d'une paire constructeur / référence articulée autour d'un **symbole** $s \in \mathcal{S}$, un nom *globalement unique* attribué une fois pour toutes à chaque définition récursive.

$\mathbf{u} ::= k$	<i>constante, primitive</i>
$\mathbf{Op}(o, \mathbf{u}, \dots, \mathbf{u})$	<i>opération n-aire</i>
$\mathbf{Rec}(s, \mathbf{u})$	$s \in \mathcal{S}$ — définition nommée
$\mathbf{Sref}(s)$	référence au symbole s

La différence cruciale : $\mathbf{Sref}(s)$ n'est **pas une variable liée** mais une constante libre. Un terme symbolique n'a pas de variables du tout — tout sous-terme est clos. Sa dénotation ne dépend d'aucun environnement, et le hash-consing y est sûr partout.

Pourquoi le hash-consing échoue sur les termes ouverts

Le hash-consing repose sur l'implication *égalité structurelle* $\implies$ *égalité sémantique*, laquelle exige que la dénotation d'un terme soit indépendante du contexte. Or la dénotation d'un terme de Bruijn ouvert est paramétrée par un environnement ρ qui interprète ses indices libres : $\llbracket \text{term} \rrbracket_\rho$. Deux pathologies symétriques en découlent.

Identification incorrecte. Considérons le terme clos

$$\mu.\text{op}(\mu.\text{op}([0], [1]), [0])$$

Les deux occurrences de $[0]$ sont structurellement identiques — le hash-consing en fait *un seul* nœud — mais l'occurrence interne réfère au μ interne et l'occurrence externe au μ externe : elles dénotent deux signaux différents. Le partage physique confond deux boucles de récursion distinctes.

Partage manqué. Symétriquement, un même signal apparaissant à des profondeurs de lieux différentes porte des indices décalés (*shifting*) : deux sous-termes sémantiquement égaux sont structurellement distincts, et le partage attendu n'a pas lieu.

Seuls les termes **clos** échappent aux deux pathologies. Mais les sous-termes d'un terme clos de Bruijn sont en général ouverts — sous un μ , le corps contient des indices libres. On ne peut donc pas hash-conser à l'intérieur des expressions récursives, c'est-à-dire précisément là où le partage rapporte le plus. D'où la nécessité de quitter la représentation de Bruijn une fois la canonicité acquise.

La transformation $\mathcal{D}$

L'idée directrice tient en une phrase : **substituer avant de descendre**. À la rencontre d'un μ , on lui attribue son symbole canonique s , on remplace dans le corps toute référence à ce μ par la constante close $S_{\text{ref}}(s)$, et seulement alors on transforme le corps — qui est redevenu clos. La clôture est ainsi un *invariant* de la traversée.

Substitution

La substitution $t\{n \mapsto u\}$ remplace l'indice n par le terme u , en incrémentant l'indice cherché sous chaque μ traversé :

$$\begin{aligned} k\{n \mapsto u\} &= k \\ \text{Op}(o, t_1, \dots, t_p)\{n \mapsto u\} &= \text{Op}(o, t_1\{n \mapsto u\}, \dots, t_p\{n \mapsto u\}) \\ [m]\{n \mapsto u\} &= \begin{cases} u & \text{si } m = n \\ [m] & \text{sinon} \end{cases} \\ (\mu.e)\{n \mapsto u\} &= \mu.(e\{n+1 \mapsto u\}) \end{aligned}$$

Noter l'absence de *shifting* de u au passage d'un lieu : la règle classique exigerait de décaler les indices libres de u , mais ici u est toujours $S_{\text{ref}}(s)$, un terme clos — le décalage est l'identité. C'est ce qui rend la substitution (et toute la passe) si simple.

Règles de transformation

La fonction $\mathcal{D}$ des termes de Bruijn **clos** vers les termes symboliques est définie par trois règles de réécriture :

$$\begin{aligned}\mathcal{D}(k) &= k \\ \mathcal{D}(\text{Op}(o, t_1, \dots, t_p)) &= \text{Op}(o, \mathcal{D}(t_1), \dots, \mathcal{D}(t_p)) \\ \mathcal{D}(\mu.e) &= \text{Rec}(s, \mathcal{D}(e\{0 \mapsto S_{\text{ref}}(s)\})) \quad \text{où } s = \Sigma(\mu.e)\end{aligned}$$

Aucune règle ne traite $\text{Ref}(n)$: sur un terme clos, un indice libre ne peut pas être rencontré — chaque indice a été substitué par la règle du μ englobant avant que la descente ne l'atteigne.

Le nommage $\Sigma : \mathcal{T}_{\text{clos}} \rightarrow \mathcal{S}$ associe un symbole à chaque terme récursif clos. Grâce au hash-consing de la représentation de Bruijn, deux définitions α -équivalentes sont *le même nœud physique* : il suffit que Σ soit une fonction de l'identité physique (une table nœud $\rightarrow$ symbole) pour que le symbole soit canonique. L' α -équivalence, résolue côté de Bruijn, est ainsi transportée dans le monde symbolique.

Proposition — Invariant de clôture

Si t est clos, alors tout appel récursif engendré par $\mathcal{D}(t)$ porte sur un terme clos. *Esquisse* — les deux premières règles ne font que descendre dans des sous-termes d'un Op , clos si t l'est privé de son éventuel μ de tête ; pour la troisième, e a au plus l'indice 0 libre, et $e\{0 \mapsto S_{\text{ref}}(s)\}$ est clos puisque $S_{\text{ref}}(s)$ l'est. Par récurrence structurelle, $\mathcal{D}$ ne rencontre jamais de terme ouvert.

Dans le terme symbolique résultant, tout sous-terme est clos : le hash-consing s'applique uniformément à la reconstruction, et le partage obtenu est maximal — y compris à travers les frontières de récursion, là où la représentation de Bruijn l'interdisait.

Exemple déroulé

Sur l'accumulateur de la 1 :

$$\begin{aligned}\mathcal{D}(\mu.+(1, [0])) &= \text{Rec}(s, \mathcal{D}(+(1, [0])\{0 \mapsto S_{\text{ref}}(s)\})) \\ &= \text{Rec}(s, \mathcal{D}(+(1, S_{\text{ref}}(s)))) \\ &= \text{Rec}(s, +(1, S_{\text{ref}}(s)))\end{aligned}$$

Et sur l'exemple pathologique de la section précédente, les deux occurrences jadis confondues de $[0]$ deviennent $S_{\text{ref}}(s_2)$ (interne) et $S_{\text{ref}}(s_1)$ (externe) — structurellement distinctes parce que sémantiquement distinctes. Le hash-

consing ne peut plus se tromper.

Mémoïsation

Telle quelle, $\mathcal{D}$ traverserait l'arbre de dérécursivation sans tenir compte du partage existant : un sous-terme de Bruijn partagé k fois serait transformé k fois, et sur un DAG profond le coût devient exponentiel en la profondeur. La fonction $\mathcal{D}$ doit donc être **mémoïsée sur l'identité physique** des nœuds de Bruijn — ce que le hash-consing rend trivial, le pointeur servant de clé. La mémoïsation joue ici un double rôle : elle ramène le coût à un parcours linéaire du DAG, et elle *préserve* le partage en garantissant qu'un nœud de Bruijn partagé produit un unique nœud symbolique.

Note

La table de mémoïsation et Σ sont deux facettes du même mécanisme : mémoïser $\mathcal{D}$ sur les nœuds $\text{Mu}(e)$ revient exactement à fixer le symbole s une fois par définition récursive physique.

Récursions mutuelles

Faust généralise μ à un lieu n -aire : un groupe de n signaux mutuellement récursifs est un unique terme $\mu^n.(e_1, \dots, e_n)$ dont on extrait les composantes par projection. La référence devient une paire (indice de lieu, numéro de projection), et la règle du μ attribue un symbole *par groupe*, les références substituées étant $\text{Sref}(s, i)$ — projection i du groupe s . La mécanique — substitution sans shifting, invariant de clôture, mémoïsation — est inchangée ; seule l'arité du nommage grandit.

References

1. Filliâtre, J.-C., Conchon, S. (2006). *Type-Safe Modular Hash-Consing*. ACM SIGPLAN Workshop on ML. [↩](#)
2. de Bruijn, N. G. (1972). *Lambda calculus notation with nameless dummies*. *Indagationes Mathematicae* 75(5), 381–392. [↩](#)